SELF-SERVICE REPORTING

Report Builder

Technical Walkthrough

~25.5 developer days (16.5 required + optional testing)

D55 Consulting

August 2026

BrytEnergy

Background

The Report Builder is a self-service report/query builder delivered as an extension to the existing Angular Customer Portal. It lets a non-technical, signed-in customer assemble a report visually: drag tables from a palette onto a flow canvas, pick the columns they want per table, connect tables into joins, and refine the design in plain language with an AWS Bedrock assistant.

When satisfied, the customer saves the report and runs it. Runs execute asynchronously against Amazon Athena, results are stored in S3, and completed results are downloadable as CSV. The goal is ad-hoc reporting over the customer's own data with no SQL and no engineering involvement.

Reports and runs are private to the signed-in customer. Because a portal user may be associated with many customer accounts, data access is scoped to the set of bryt numbers the user is authorised for - resolved server-side by replicating the Customer Portal's customer-access logic - and every generated query is pinned to that set using trusted server-side context.

> ### The shape of the work, in one line
>
> A visual canvas and a Bedrock assistant edit one shared, logical report design; a pure generator turns that design into bounded, bryt-number-pinned Athena SQL; and an independent verifier re-checks the scoping before execution and again over every result row. Security is the spine, not a layer on top.

How it works

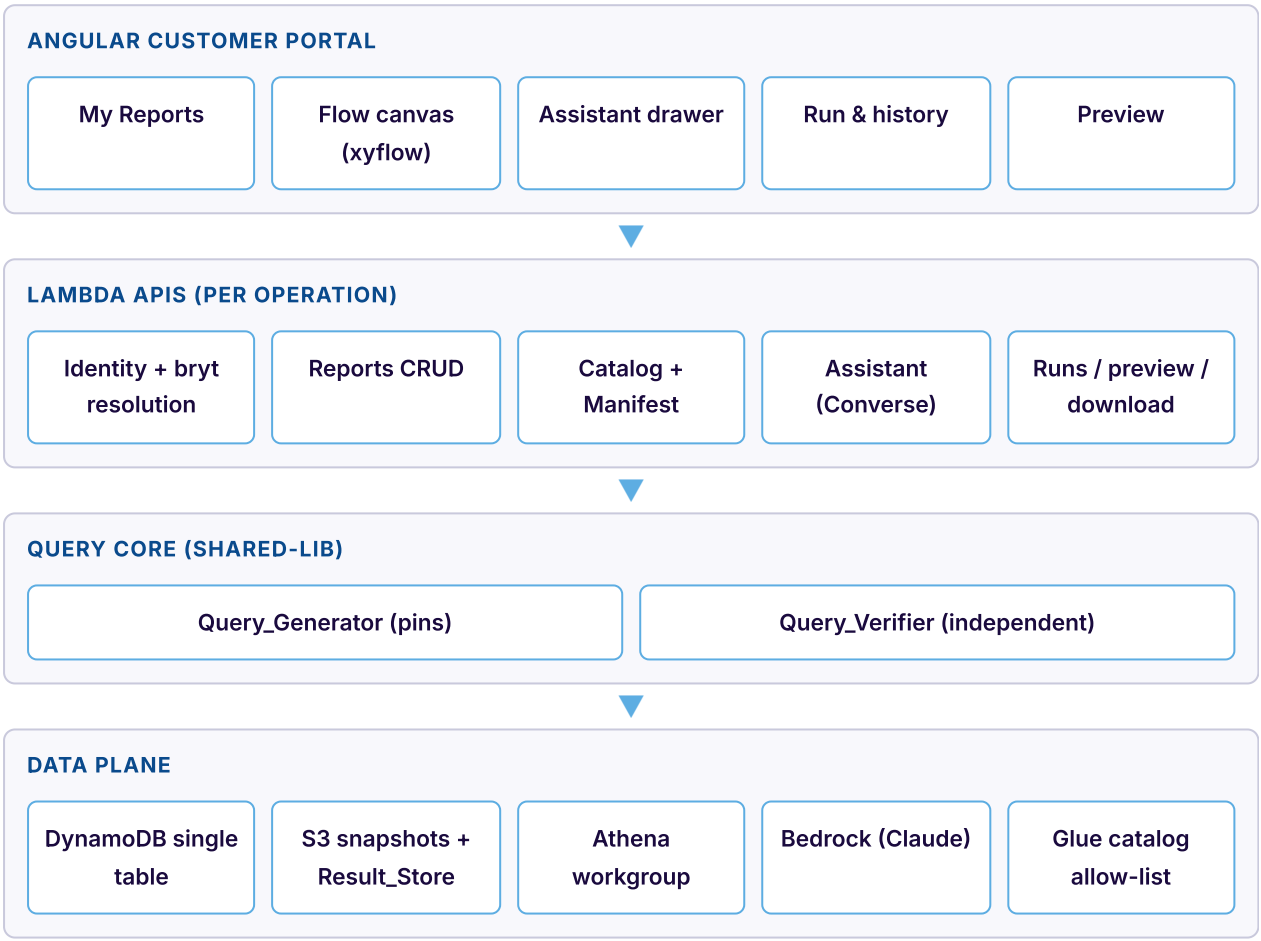
The backend is a new repository, BrytReportBuilder, mirroring the established BrytBusinessServices (contract-note) patterns: per-operation TypeScript Lambda handlers grouped by domain folder, a single DynamoDB table, versioned/encrypted S3 buckets, an API Gateway REST resource tree, Step Functions for the async run pipeline, and a shared library for shared types.

A report is captured as a Report_Design - a logical description of intent (selected tables, columns, joins, filters, sort), never SQL. The same design instance is edited by both the flow canvas and the assistant, so visual and conversational edits stay in sync. SQL is derived from the design only at run or preview time, by the Query_Generator, keeping the security-sensitive translation in one audited place.

The Query_Generator and Query_Verifier live in the shared library because they run in three contexts - assistant validation, preview, and the run pipeline - and are the security spine. One implementation, one place to test.

System architecture

The Angular portal feature talks to a set of per-operation Lambda APIs. Those APIs resolve identity and the authorised bryt numbers first, then read and write the single DynamoDB table and the S3 buckets. The assistant calls Bedrock; the query core (generator + verifier) and the Step Functions pipeline drive Athena.



Every API resolves identity and the authorised bryt numbers before any data work; the query core and pipeline are the only paths that reach Athena.

Asynchronous run pipeline

Queuing a run writes a Queued record, starts a Step Functions state machine, and returns a run id immediately. The pipeline generates the pinned SQL, verifies the pin and bounds before execution, runs Athena, writes the CSV to the Result_Store, then re-verifies every result row before the run is marked Complete. A catch on every state routes failures to a single handle-failure state that records the error and discards any partial output.

Verify runs twice - statically before execution and over the result set before download. A blocked query or a foreign result row marks the run Failed with no downloadable output.

Key design decisions

The design settled a handful of choices (grounded in the Phase 0 decision artifacts) that shape the build.

DECISION	CHOICE	WHY
Bedrock integration	Roll-our-own Converse API tool-use loop in a Lambda	We own the loop, trusted-context injection, audit logging, and the forced validate step; the verifier stays outside the model
Dry-run validation	Athena EXPLAIN	Validates SQL and resolves catalog metadata with no data scan, and is not charged
Report_Design persistence	Primary in DynamoDB, versioned JSON snapshot in S3 on save	Small JSON gives fast owner-scoped list/CRUD; the S3 snapshot gives history and satisfies the versioned-storage requirement
Async run	Step Functions generate → verify → execute → write → finalise, catch on every state	Returns a run id before completion; isolates and reports failure at each stage
Bryt pin	bryt_number IN (:authorised_bryt_numbers) from trusted context	Supports multi-account users; single-account narrowing passes a one-element subset that must be in the set
via-mpan pinning	Join to a supply_mpan mapping with an effective-date window	mpan → bryt is many-over-time (change of tenancy); the window prevents cross-tenant leakage. The verifier rejects unjoined/window-less reads
Catalog source	Curated allow-list intersected with Glue, fail-closed	Role-visible metadata is not the same as queryable; IAM and Lake Formation are separate, per-environment grants
Preview	Server-side bounded Athena query (LIMIT 100), same generate→verify path	A bounded, pinned, verified query is mandated; a client-side sample would be a security regression

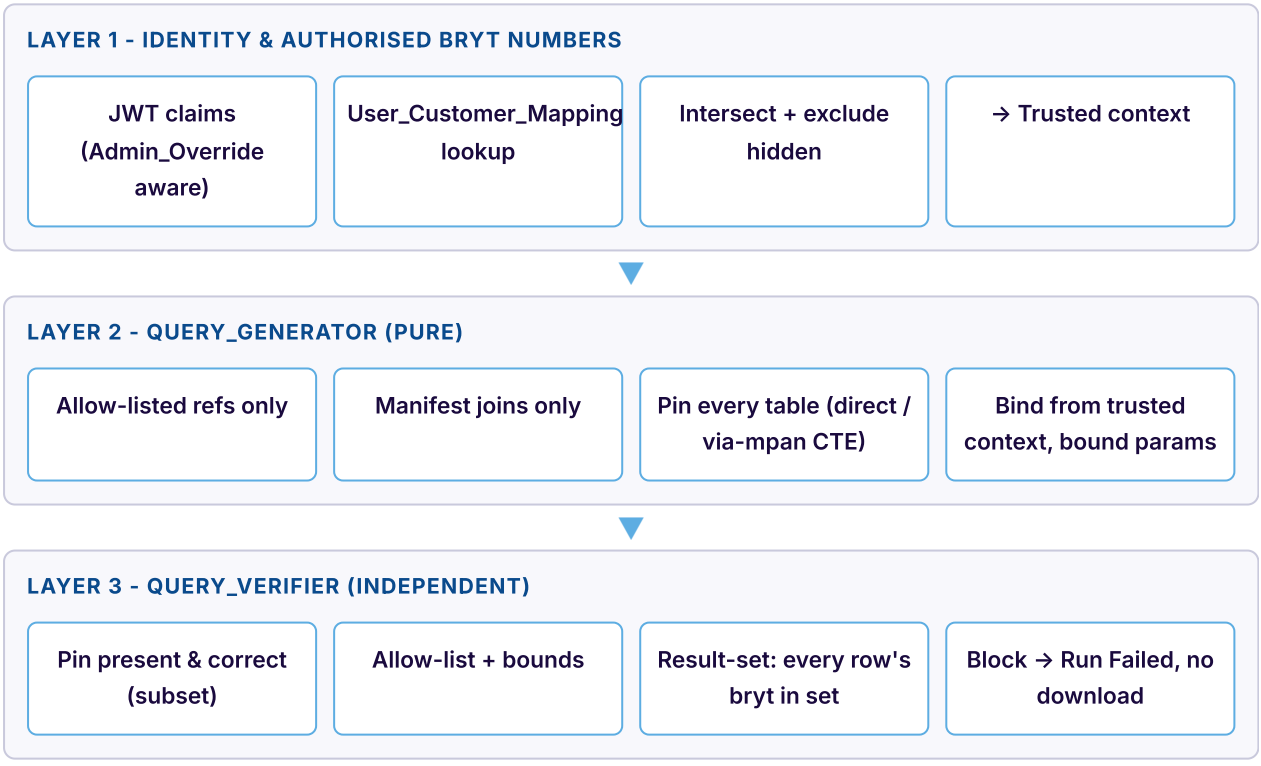
DECISION	CHOICE	WHY
Query bounds	Run 100k rows / 50 GiB; preview 100 rows / 1 GiB; configurable	The byte bound is the real cost control; the Athena workgroup enforces a hard cutoff as defence-in-depth

The security spine

Data isolation is enforced by three independent layers, and no single layer is trusted alone. Even a fully-compromised assistant cannot leak data, because the generator only ever binds the pin from trusted context and the independent verifier blocks any query or result that violates the pin or the bounds.

Three independent layers

Identity resolves server-side to the authorised bryt numbers, which become trusted context. The pure generator pins every table from that trusted context only. The independent verifier re-derives its expectations from trusted context plus the manifest, so it cannot be satisfied by anything the model emitted.



The verifier is not a model tool and not part of generation. It runs before execution and again after completion.

Prompt-injection defence

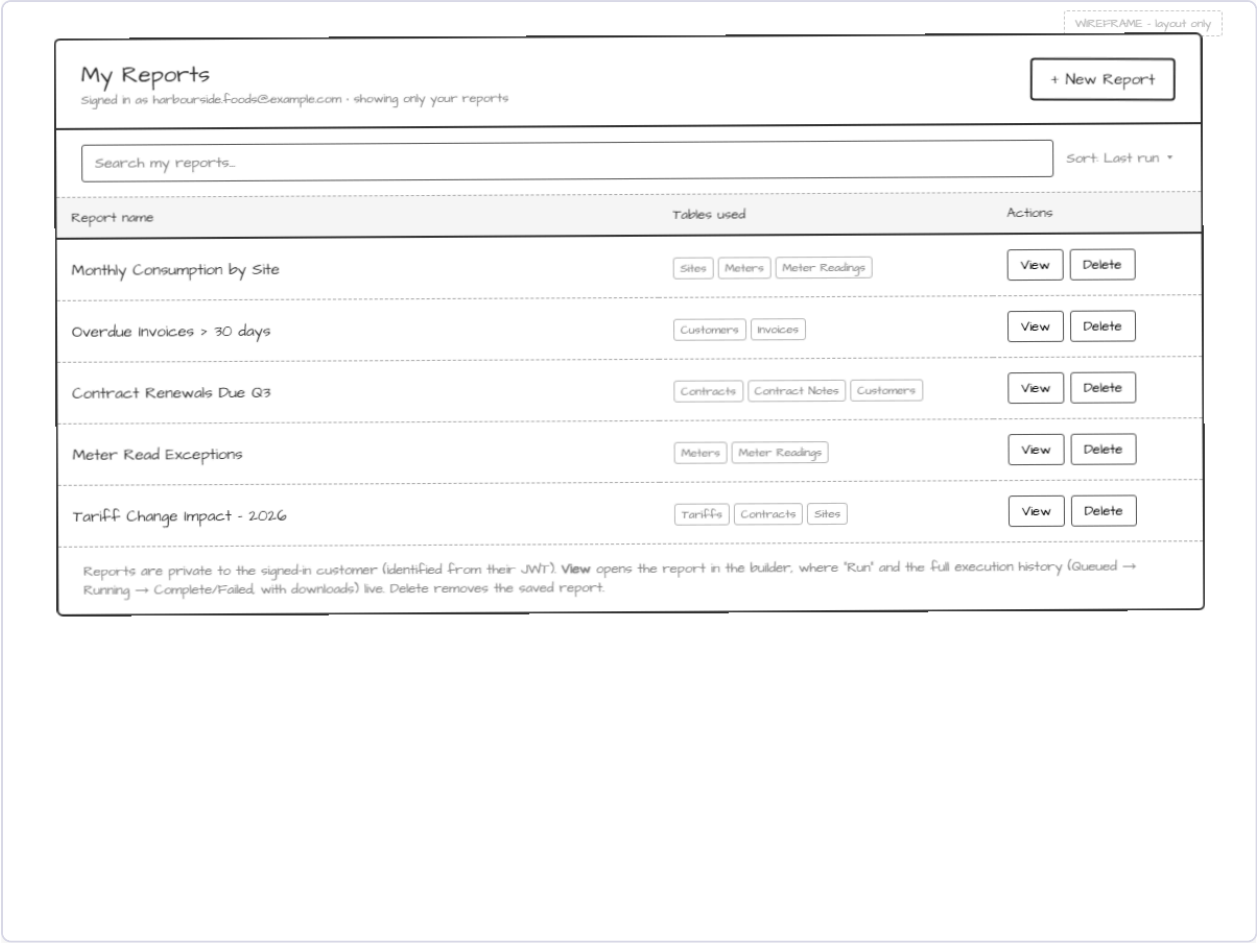
The assistant treats all prompt content and any data pulled into context as untrusted input.

THREAT	RESPONSE
Instruction hidden in a prompt or in data to drop the bryt-number scoping	Ignored; the enforced constraints are preserved and the request completes using only trusted-context scoping
Assistant coaxed into emitting a query with no pin, a disallowed table, or over-bounds	Caught by the independent verifier, which enforces pin/allow-list/bounds regardless of model output
Manipulation attempt detected in untrusted input	An audit entry is recorded noting the attempt was ignored
Bryt-number filter derived from model output or user prompt	Never happens - the filter is bound only from trusted context, and filter values are always bound parameters

Screen-by-screen walkthrough

Seven screens make up the customer experience, from the private reports list through the visual builder, assistant, preview, run history, and save. Each is shown below with the interactions it supports and what happens behind the screen.

1. My Reports



The entry point: the signed-in customer's private list of saved reports, each showing the tables it uses as badges, with search and A-Z / Z-A sort.

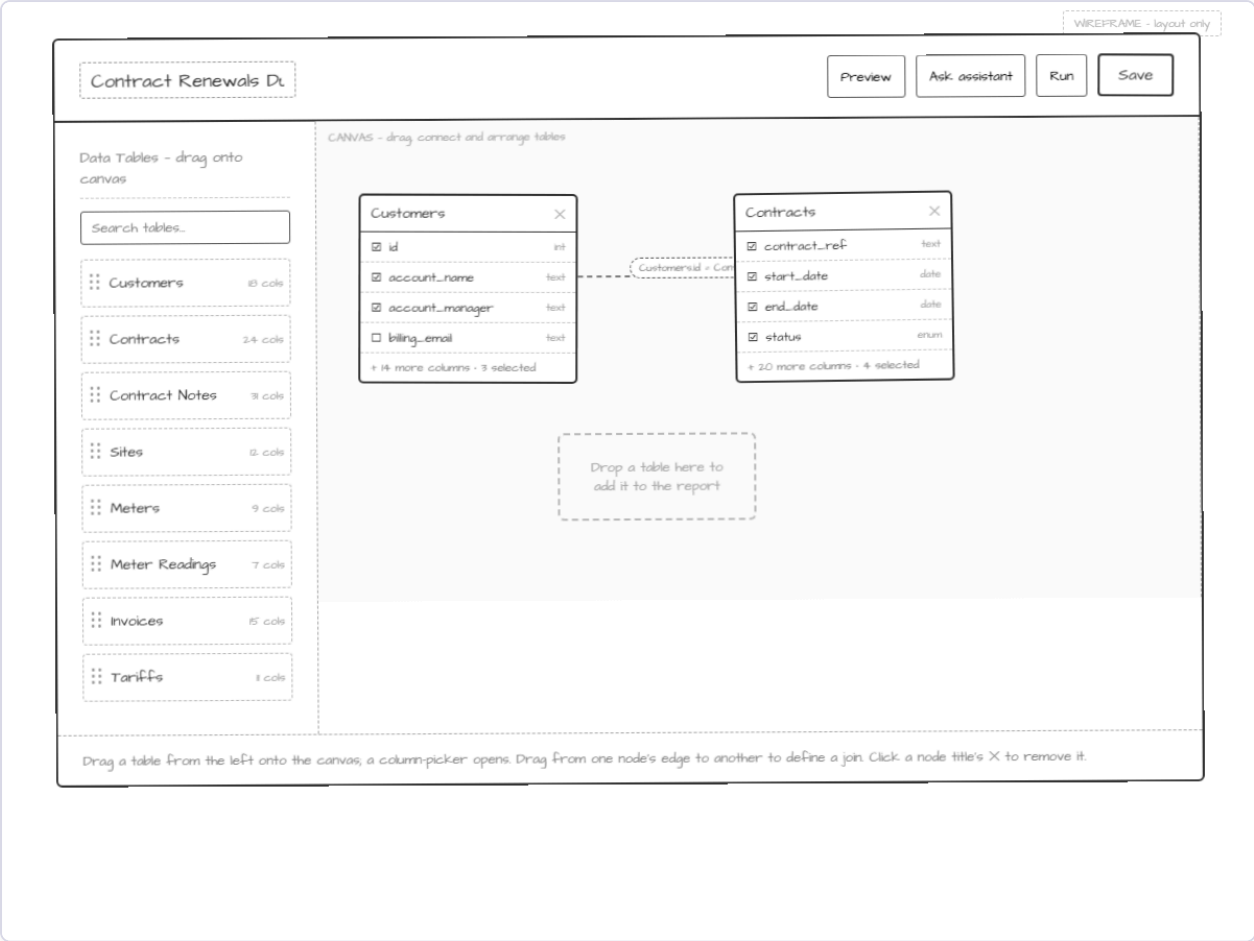
KEY INTERACTIONS

- + New Report opens the builder with an empty design
- View opens a saved report's design in the builder
- Delete prompts for confirmation before removing
- Search filters by name (case-insensitive substring); sort orders A-Z or Z-A
- Empty state invites the user to create their first report

BEHIND THE SCREEN

- Lists only reports owned by the effective identity (Report items on USER#<effectiveId>)
- Sort/search are served by the name GSI (GSI1SK = NAME#<lowername>)
- An invalid/expired JWT shows a re-authentication error and no reports
- A failed load offers a retry; a failed delete keeps the report in the list

2. Builder Canvas



The visual designer. A searchable palette of allow-listed tables sits on the left; dragging a table onto the canvas opens the column picker, and dragging between node edges creates a join using the manifest predicate for that table pair.

KEY INTERACTIONS

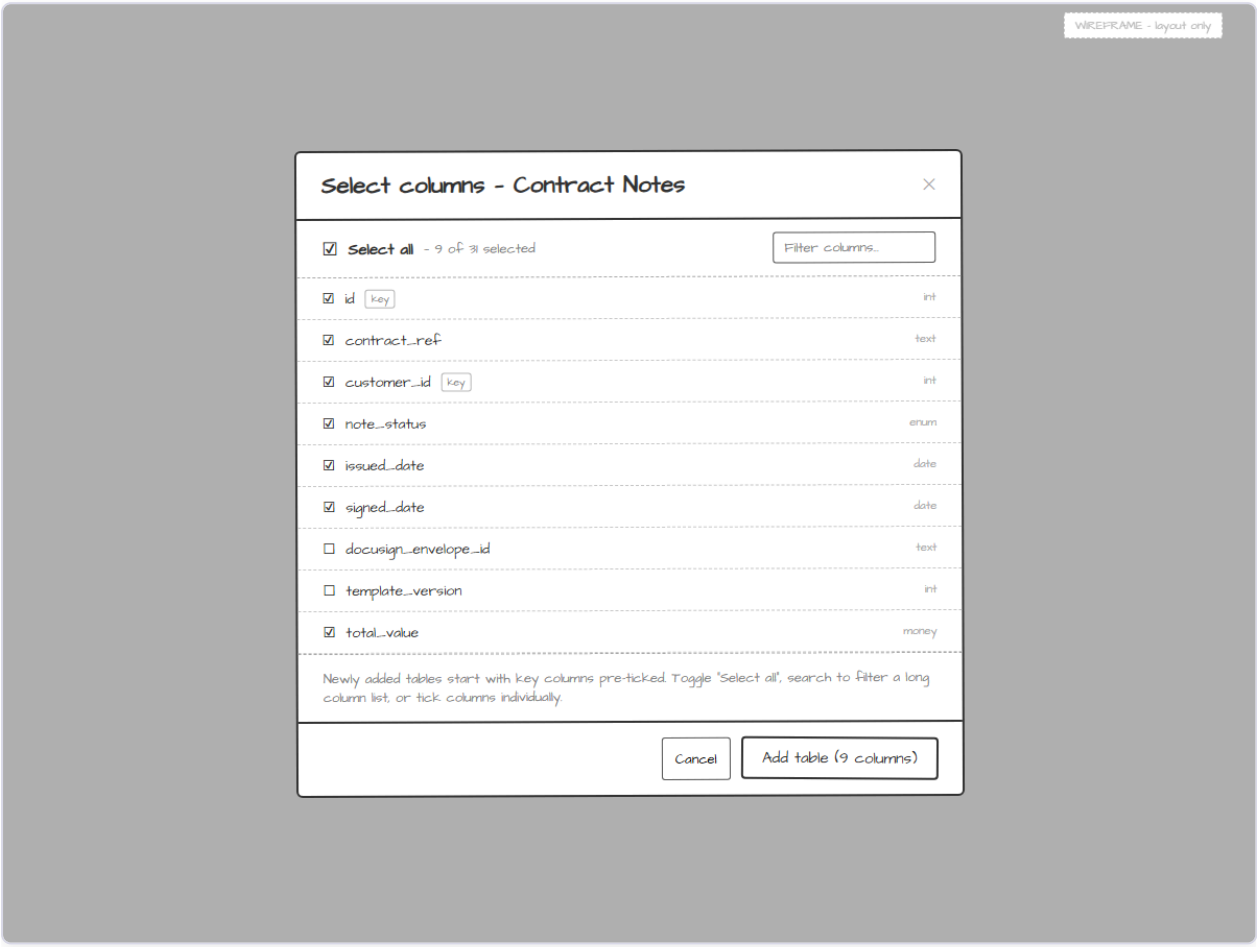
- Drag a table from the palette → opens the column picker
- Drag from one node's edge to another -> creates a manifest-defined join
- A join with no manifest predicate is rejected with a message
- Remove a node to drop the table and its joins from the design
- Edit the report name (1-200 chars) in the header

BEHIND THE SCREEN

- Each node is one SelectedTable; each edge is one DesignJoin (1:1 graph mapping)
- Nodes show selected columns and an 'X of N selected' summary
- Join lines carry the join-condition badge from the manifest predicate
- Every edit runs validateDesign against the catalog + manifest

- Header actions: Preview, Ask assistant, Run, Save

3. Column Picker



Opened when a table is added. Lists all allow-listed columns with type and a key tag on join/primary keys; key columns are pre-selected, everything else starts unselected.

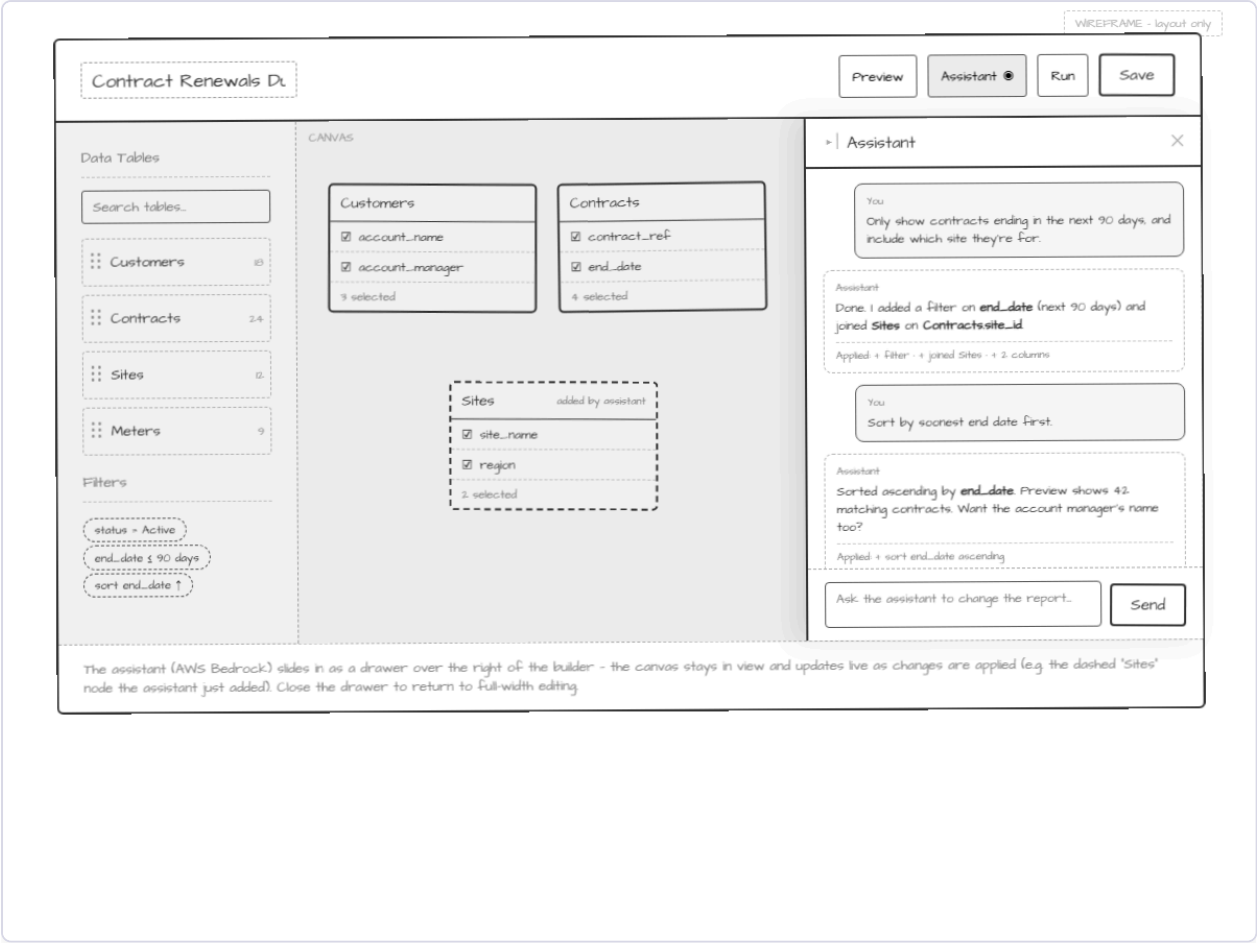
KEY INTERACTIONS

- Toggle individual columns; 'X of N selected' updates live
- Select all (respects an active filter - selects only displayed columns)
- Filter by column name (case-insensitive substring)
- Add table confirms with at least one column selected

BEHIND THE SCREEN

- Columns come from the catalog allow-list, with type and isKey flags
- Key columns (join/primary) are pre-selected on first open
- Confirming with zero columns is blocked with a message
- A column-load failure offers a retry

4. Assistant Drawer



A Bedrock-backed helper that reads and edits the same Report_Design in response to plain-language requests, describing each change it applies. The canvas and palette stay visible alongside it.

KEY INTERACTIONS

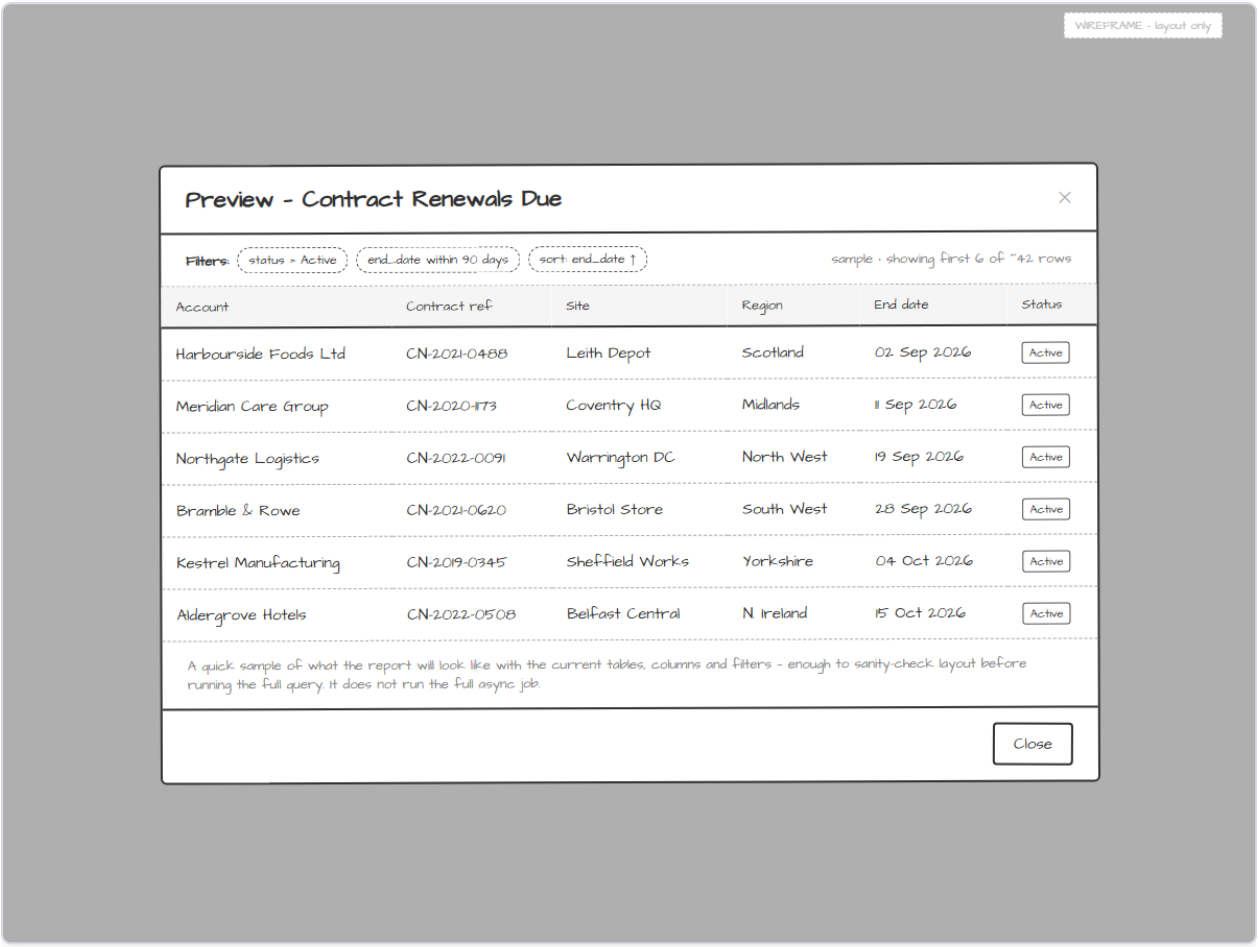
- Submit a 1-2000 character message; empty/whitespace/oversize is rejected client-side
- Applied changes update the shared design and reflect on the canvas within ~2s
- The assistant returns a per-change description and an applied-change summary
- Requests outside the catalog/manifest are declined with the specific limitation

BEHIND THE SCREEN

- Mutation tools (add_table, add_join, set_filter, …) run through the same validateDesign
- validate_query is forced (via toolChoice) before an applied change finalises
- Trusted context (authorised bryt numbers, manifest) is injected by the Lambda, never from model output
- Conversation history persists per report + owner and restores on reopen

- Closing the drawer returns to full-width editing with the design retained

5. Preview



A lightweight sample of at most 100 rows to sanity-check layout before running the full job. Preview does not queue a run.

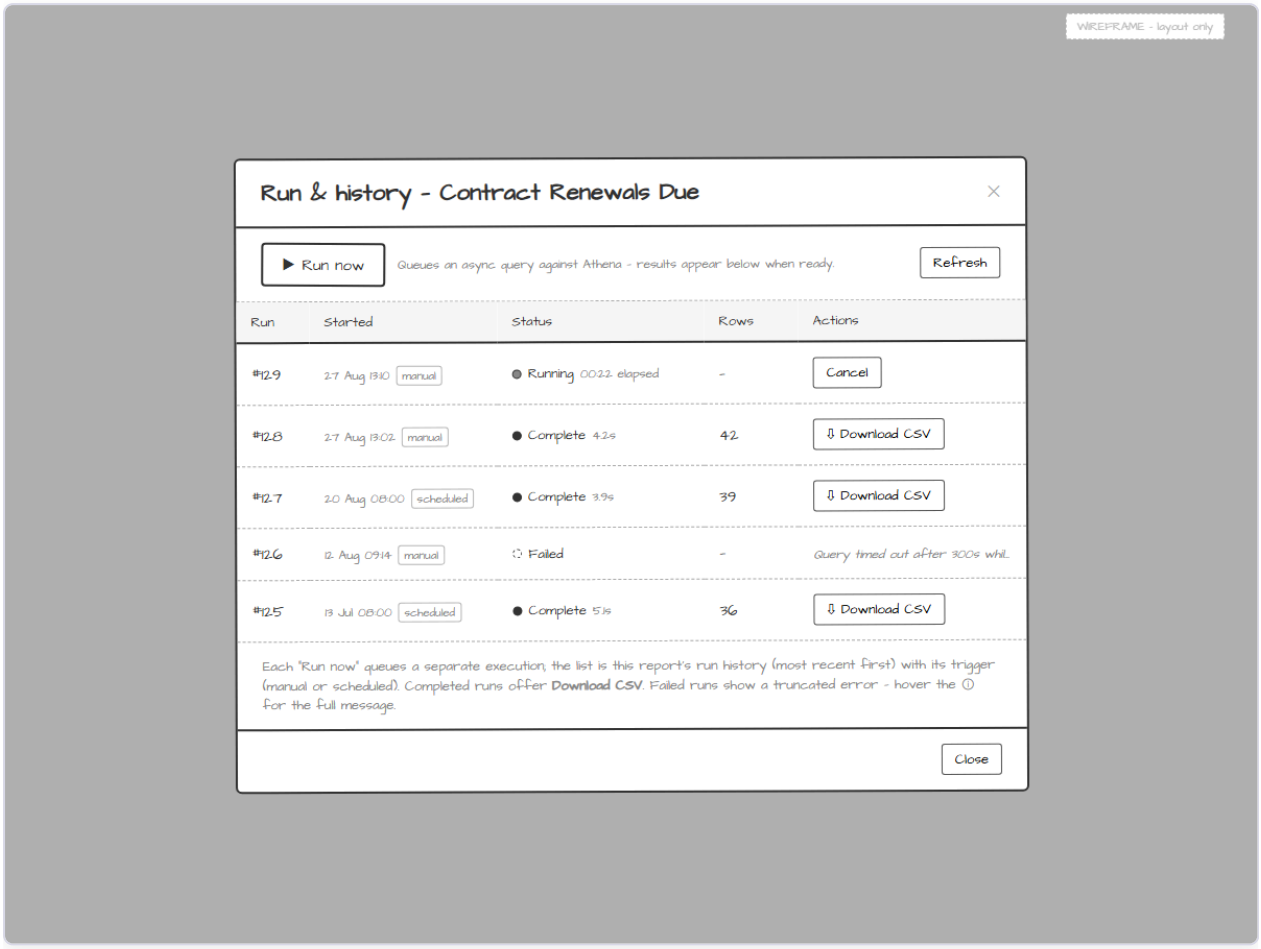
KEY INTERACTIONS

- Preview shows up to 100 rows, only the selected columns, in design order
- A filter and sort summary strip lists the active rules
- A progress indicator shows while generating; Close returns to the builder
- Zero rows shows an empty-result indication with columns + summary still shown

BEHIND THE SCREEN

- Runs the same generate → verify path with preview bounds (100 rows / 1 GiB), pinned
- EXPLAIN runs in front to fail fast; a 10s budget bounds the attempt
- No asynchronous run is queued or started
- Failure or timeout shows an error and leaves the design unchanged

6. Run & History



Queues asynchronous runs and lists this report's recent runs with status, timing, row count, and per-run actions.

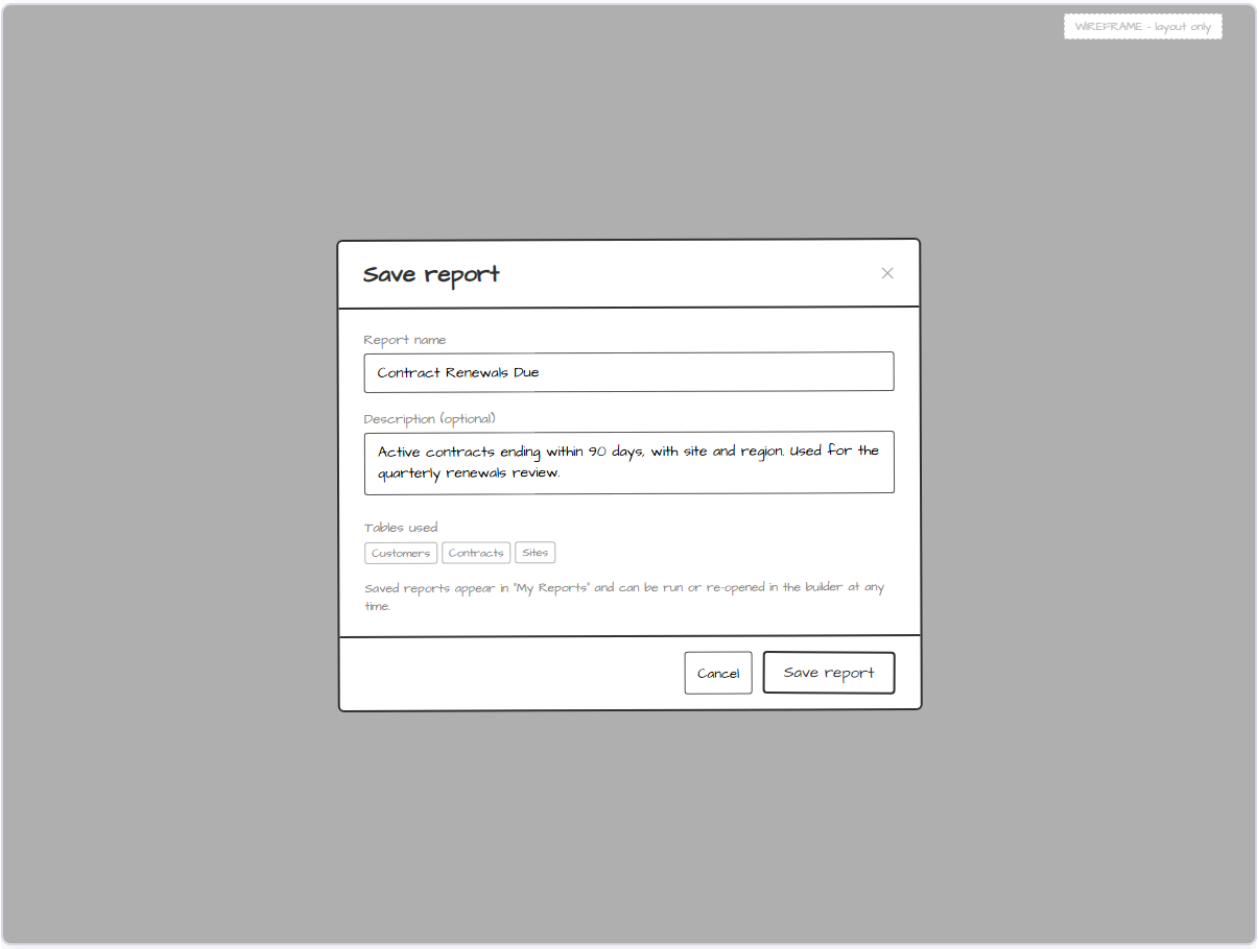
KEY INTERACTIONS

- Run now queues a run and shows a Queued row at the top within ~3s
- Up to 50 most-recent runs, newest first, with run number, started time, trigger, status, row count
- Complete runs offer Download CSV; failed runs show a truncated error (full on hover)
- Queued/Running runs offer Cancel; Refresh re-reads current statuses

BEHIND THE SCREEN

- Run items are keyed for recency via GSI2 (STARTED#<ts>)
- Download is a short-lived pre-signed URL to the owner-scoped Result_Store key
- Cancel stops the Step Functions execution and the Athena query; terminal states reject it
- A missing result object surfaces a retrieval error

7. Save Report



Persists the design under a name (1-100 chars) and an optional description (up to 500), with the tables used shown as badges.

KEY INTERACTIONS

- Confirm save with a valid trimmed name persists within ~5s
- Empty/whitespace name or over-length name/description is blocked with a message
- A saved report appears in My Reports
- Saving an existing report updates it in place - no duplicate entry

BEHIND THE SCREEN

- The serialised Report_Design is stored inline on the Report item, scoped to the effective identity
- A versioned JSON snapshot is written to S3 on each save for history
- A persistence failure retains the form values and design and shows a message

Data model at a glance

Everything lives in a single DynamoDB table scoped by the effective identity, with the larger blobs (design snapshots, result CSVs) in S3. Full record shapes are in the companion Data Model document.

ENTITY	KEY PATTERN	HOLDS
Report	USER#<effectiveId> / REPORT#<reportId>	Serialised Report_Design inline + metadata (name, description, table list, timestamps)
Conversation message	USER#<effectiveId> / REPORT#<reportId>#MSG#<ts>	Per-report assistant history, time-ordered
Run	USER#<effectiveId> / REPORT#<reportId>#RUN#<runNo>	Run number, trigger, status, row count, error, Result_Store location
Design snapshot (S3)	snapshots/<effectiveId>/<reportId>.json	Versioned JSON snapshot on each save; history/restore, off the read path
Result CSV (S3)	results/<effectiveId>/<reportId>/<runNo>.csv	Run output; a disabled lifecycle rule reserves expiry for later

API surface

The Report_API mirrors the contract-note handler pattern. Every handler resolves identity and the authorised bryt numbers first and scopes all store access to the effective identity.

METHOD	ROUTE	PURPOSE
GET / POST	/reports	List the owner's reports; create a report
GET / PUT / DELETE	/reports/{reportId}	Read, update-in-place, delete a report

METHOD	ROUTE	PURPOSE
GET	/catalog & /catalog/manifest	Fail-closed allow-listed catalog; read-only Join_Manifest
POST	/reports/{reportId}/assistant	Assistant Converse turn over the shared design
POST	/reports/{reportId}/preview	Synchronous bounded preview (no run queued)
POST / GET	/reports/{reportId}/runs	Queue a run; list up to 50 most-recent runs
GET	/reports/{reportId}/runs/{runId}	Run status
POST	/reports/{reportId}/runs/{runId}/cancel	Cancel a Queued/Running run
GET	/reports/{reportId}/runs/{runId}/result	Download the verified CSV (pre-signed URL)

Open questions and assumptions

The Phase 0.5 decisions closed the original open questions; these are the working assumptions carried into the build. One item remains deferred.

AREA	WORKING ASSUMPTION
Preview execution	Server-side bounded Athena query (LIMIT 100) via the same generate->verify path, not a client sample
'View' on My Reports	Opens the report in the builder; runs and results live in the Run & history modal
CSV retention	Retained indefinitely for MVP; a disabled S3 lifecycle rule reserves expiry
Run-completion notifications	In-portal polling and Refresh only; no email/push for MVP
Query-bound defaults	Run 100k rows / 50 GiB, preview 100 rows / 1 GiB, all configurable
MVP catalog allow-list	The 9 dev-verified tables; ecoes_activity and both Jira tables excluded (fail-closed)
Deferred	Prod value-verification of the mpan mapping + medium-confidence joins (needs a scoped Lake Formation grant on prod)

Delivery breakdown

The build is grouped into eight phases. The security spine (Phase 2) is sequenced before any execution path - no query runs before the verifier exists and is property-tested. Day figures are indicative, derived from the 38-task plan.

PHASE	SCOPE	DAYS
Repo & shared-lib foundations	Repo scaffold (api/, cdk/, shared-lib/), domain types, Join_Manifest promotion, shared HTTP + identity helpers	1.2
Security spine	The security spine: validateDesign, identity/bryt resolution, Query_Generator, Query_Verifier, serialise round-trip, spine property tests	4.2

PHASE	SCOPE	DAYS
CDK foundation	CDK: single DynamoDB table, versioned/encrypted S3 buckets, REST API + JWT authorizer, per-env IAM + Lake Formation role, Athena workgroup, health deploy	0.8
Catalog + Reports CRUD	Catalog service (fail-closed allow-list) + Join_Manifest endpoint; owner-scoped Reports CRUD; versioned S3 design snapshot on save	0.8
Assistant (Converse loop) + query generation	Assistant Converse loop, Report_Design mutation tools, forced validate_query (EXPLAIN), conversation persistence, prompt-injection defence + audit	6.8
Run pipeline + preview + download	Step Functions run pipeline, run queue/status/list, cancel + terminal guard, CSV download, synchronous preview	3.2
Frontend (Angular Customer Portal extension)	Angular extension: flow-canvas spike, shared client Report_Design + graph mapping, the seven screens	5
Hardening, tests, deploy	Cross-tenant/injection/bounds security test suite, observability, CI/CD, end-to-end walkthrough + sign-off	3.5
Total	38 tasks across 8 phases	25.5

Testing note

The build is designed around 13 correctness properties - every query pinned; the verifier blocks unpinned or out-of-bounds queries; no foreign-bryt result reaches the user; the effective-date window prevents cross-tenancy leakage on via-mpan reads; the design round-trips; only allow-listed references are accepted; identity is server-resolved. Property-based, integration, and security tests are marked optional in the plan and can be deferred for a faster MVP, but they are strongly recommended given the feature's data-isolation guarantees.